```
# ============================================================
```

CS-308-MJ -P DATASCIENCE ANDANALYTICS

```
# ALL 25 PRACTICAL SLIPS - COPY FORMAT

# ============================

# ============================
```

SLIP 1

```
# ============================
```

Q1- Product Dendrogram

```
import pandas as pd

import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

df =pd.DataFrame({

    "Price":[100, 120, 500, 520, 110 ],

    "Rating":[4.1, 4.0, 4.8, 4.7, 4.2],

    "Sales":[90, 85, 35, 40, 95]
})
z =linkage(df, method="ward")
dendrogram(z)
plt.title("Product Dendrogram")

plt.xlabel("Products")
plt.ylabel("Distance")

plt.show()
```

Q2 - LoanData Reduction

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.feature_selectionimport SelectKBest, chi2

from sklearn.decompositionimport PCA

df =pd.DataFrame({

    "Age":[25, 30, 35, 40, 45, 50, 28, 32, 38, 42],

    "Income":
[25000, 35000, 45000, 55000, 65000, 75000, 30000, 40000, 50000, 60000 ],
    "Credit_Score":[600, 650, 680, 700, 720, 750, 620, 660, 690, 710 ],

    "Loan_Amount":
[100000, 150000, 200000, 250000, 300000, 350000, 120000, 180000, 220000,
280000 ],
    "Employment_Years":[2, 4, 6, 8, 10, 12, 3, 5, 7, 9],

    "Debt_Ratio":[.5, .4, .35, .3, .25, .2, .45, .38, .32, .28],

    "Loan_Status":["Rejected", "Rejected", "Approved", "Approved", "Approved",

    "Approved", "Rejected", "Approved", "Approved", "Approved"]
})
x =df.drop("Loan_Status", axis=1)

y =LabelEncoder().fit_transform(df["Loan_Status"])

sel =SelectKBest(chi2, k=3)
sel.fit(x, y)
print("Selected Features:")
print(x.columns[sel.get_support()].tolist())
```

```
x =StandardScaler().fit_transform(x)

pca =PCA(n_components=2)

print("PCAResult:")

print(pca.fit_transform(x))
```

ORQ2 - Iris K-Means K=3

```
from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

iris =load_iris()

x =iris.data

model =KMeans(n_clusters=3, random_state=42, n_init=10 )

labels =model.fit_predict(x)

plt.scatter(x[:, 0 ], x[:, 1], c = labels)
plt.scatter(model.cluster_centers_[:, 0 ],

    model.cluster_centers_[:, 1],

    marker="X", s=200 )

plt.xlabel("SepalLength")

plt.ylabel("SepalWidth")

plt.title("Iris K-Means")

plt.show()

print(model.cluster_centers_)

# ========================
    ==
```

SLIP 2

```
# ========================
```

Q1- EqualWidthBinning

```
import pandas as pd

df =pd.DataFrame({

    "Person_ID":range(1, 11),

    "Age":[18, 22, 25, 30, 35, 40, 45, 50, 55, 60 ]
})
df["Age_Bin"] =pd.cut(

    df["Age"],
    bins=3,

    labels =["Young", "Middle", "Old"]
)
print(df)
```

Q2 - YouTube Linear Regression

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.linear_modelimport LinearRegression

df =pd.DataFrame({

    "Likes":[100, 200, 300, 400, 500, 600, 700, 800 ],

    "Views":[1000, 1800, 2900, 3900, 5200, 6100, 7000, 8200 ]
```

```
})
 x =df[["Likes"]]

 y =df["Views"]

 model =LinearRegression()

 model.fit(x, y)

 pred =model.predict(x)
 plt.scatter(x, y)

 plt.plot(x, pred)

 plt.xlabel("Likes")

 plt.ylabel("Views")

 plt.title("YouTube Linear Regression")

 plt.show()

 print(pred)
```

ORQ2 - Apriori

```
 from mlxtend.preprocessing import TransactionEncoder

 from mlxtend.frequent_patterns import apriori, association_rules

 import pandas as pd

 data =[

     ["Video Lecture", "Quiz"],

     ["Video Lecture", "Assignment"],

     ["Quiz", "DiscussionForum"],

     ["Video Lecture", "Quiz", "Assignment"],

     ["Assignment", "DiscussionForum"],

     ["Video Lecture", "Certificate"],

     ["Quiz", "Certificate"],

     ["Video Lecture", "Quiz"],
     ["Assignment", "Certificate"],

     ["Video Lecture", "Quiz", "Certificate"]
]
 te =TransactionEncoder()

 arr =te.fit(data).transform(data)

 df =pd.DataFrame(arr, columns = te.columns_)

 for s in[0 .2, 0 .3]:

     print("Support =", s)

     freq =apriori(df, min_support = s, use_colnames = True)

     print(freq)

     if len(freq) >1:

     print(association_rules(

     freq,

     metric="confidence",

     min_threshold=0 .5

     ))

 # ========================
```

SLIP 3

```
# ==============
    ==========
```

Q1- Vehicle Box Plot

```
import pandas as pd

import matplotlib.pyplot as plt

df =pd.DataFrame({

    "Fuel_Efficiency":[20, 22, 24, 25, 60 ],

    "Engine_Power":[100, 120, 130, 140, 300 ],

    "Vehicle_Weight":[1200, 1300, 1400, 1500, 1600 ],

    "Engine_Size":[1000, 1200, 1400, 1600, 1800 ],

    "Acceleration":[12, 11, 10, 9, 8]
})
for colindf.columns:

    plt.boxplot(df[col])

    plt.title(col)

    plt.show()
```

Q2 - Salary Linear Regression

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selectionimport train_test_split

from sklearn.linear_modelimport LinearRegression

from sklearn.metrics import mean_squared_error
df =pd.DataFrame({

    "YearsExperience":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],

    "Salary":[30000, 35000, 40000, 45000, 50000,

    60000, 65000, 70000, 80000, 90000 ]
})
x =df[["YearsExperience"]]

y =df["Salary"]
x_train, x_test, y_train, y_test =train_test_split(
    x, y, test_size=.2, random_state=42
)
model =LinearRegression()

model.fit(x_train, y_train)

pred =model.predict(x_test)

print("MSE:", mean_squared_error(y_test, pred))

plt.scatter(x, y)

plt.plot(x, model.predict(x))

plt.show()
```

ORQ2 - Heart Disease RandomForest

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix
```

```
df =pd.DataFrame({
    "Age":[45, 50, 55, 60, 65, 40, 48, 52, 58, 62],
    "Cholesterol":[200, 220, 250, 270, 290, 180, 210, 230, 260, 280 ],
    "BP":[120, 130, 140, 150, 160, 110, 125, 135, 145, 155],
    "Target":[0, 0, 1, 1, 1, 0, 0, 1, 1, 1]
})
x =df.drop("Target", axis=1)
y =df["Target"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
model =RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(x_train, y_train)
pred =model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("ConfusionMatrix:")
print(confusion_matrix(y_test, pred))
# ============================================================
```

SLIP 4

```
# ============================================================
```

Q1- Payment Method Pie Chart

```
import matplotlib.pyplot as plt
payment = ["UPI", "Credit Card", "Debit Card", "Cash", "Net Banking"]
customers = [150, 80, 60, 40, 30 ]
plt.pie(customers, labels =payment, autopct="%1.1f%%")
plt.title("Preferred Payment Methods")
plt.show()
```

Q2 - Online Gaming Logistic Regression

```
import pandas as pd
from sklearn.model_selectionimport train_test_split
from sklearn.linear_modelimport LogisticRegression
from sklearn.metrics import
accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
df =pd.DataFrame({
    "Play_Time":[1, 2, 2, 3, 4, 5, 6, 7, 8, 9],
    "Achievements":[2, 3, 4, 5, 6, 7, 8, 9, 10, 12],
    "Purchases":[0, 1, 0, 1, 2, 2, 3, 3, 4, 5],
    "Level":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
    "Class":[0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
})
x =df.drop("Class", axis=1)
y =df["Class"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
model =LogisticRegression()
model.fit(x_train, y_train)
```

```
pred =model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, pred))

print("Precision:", precision_score(y_test, pred, zero_division=0 ))

print("Recall:", recall_score(y_test, pred, zero_division=0 ))

print("F1:", f1_score(y_test, pred, zero_division=0 ))

print("ConfusionMatrix:")

print(confusion_matrix(y_test, pred))
```

ORQ2 - E-Commerce K-Means

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

df =pd.DataFrame({

    "Annual_Income":[15, 20, 25, 30, 35, 50, 55, 60, 70, 80 ],

    "Spending_Score":[80, 75, 70, 65, 60, 45, 40, 35, 25, 20 ]
})
x =df[["Annual_Income", "Spending_Score"]]

model =KMeans(n_clusters=3, random_state=42, n_init=10 )

df["Cluster"] = model.fit_predict(x)

print(df)
plt.scatter(

    df["Annual_Income"],

    df["Spending_Score"],

    c = df["Cluster"]
)
plt.xlabel("AnnualIncome")

plt.ylabel("Spending Score")

plt.show()

# ============================================================
```

SLIP 5

```
# ============================================================
```

Q1- LabelEncoding

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

df =pd.DataFrame({

    "Customer_ID":[1, 2, 3, 4, 5],

    "Gender":["Male", "Female", "Male", "Female", "Male"],
    "City":["Pune", "Nashik", "Mumbai", "Pune", "Nashik"],

    "Membership_Type":["Gold", "Silver", "Gold", "Bronze", "Silver"]
})
le = LabelEncoder()

for colin["Gender", "City", "Membership_Type"]:

    df[col] = le.fit_transform(df[col])

print(df)
```

Q2 - Netflix/ AmazonVenn+Pie

```
import matplotlib.pyplot as plt

from matplotlib_vennimport venn2

netflix={"A", "B", "C", "D", "E"}

amazon={"C", "D", "E", "F", "G"}

only_netflix = len(netflix-amazon)

only_amazon = len(amazon-netflix)

both = len(netflix&amazon)
venn2([netflix, amazon], set_labels = ("Netflix", "Amazon"))
plt.show()

plt.pie(

    [only_netflix, only_amazon, both],

    labels =["Netflix", "Amazon", "Both"],

    autopct="%1.1f%%"
)
plt.show()
```

ORQ2 - Spambase KNN

```
from sklearn.datasets import make_classification

from sklearn.model_selectionimport train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix
x, y =make_classification(
    n_samples=300,

    n_features=10,

    n_classes=2,

    random_state=42
)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
scaler =StandardScaler()

x_train = scaler.fit_transform(x_train)

x_test = scaler.transform(x_test)

model =KNeighborsClassifier(n_neighbors=5)

model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print("ConfusionMatrix:")

print(confusion_matrix(y_test, pred))

# ============================================================
```

SLIP 6

```
# ============================================================
```

Q1- Car Scatter Plot

```
import pandas as pd

import matplotlib.pyplot as plt
```

```
df =pd.DataFrame({
    "Engine_Size":[800, 1000, 1200, 1500, 1800 ],
    "Price":[200000, 300000, 400000, 550000, 700000 ]
})
plt.scatter(df["Engine_Size"], df["Price"])
plt.xlabel("Engine Size")
plt.ylabel("Price")
plt.title("Car Engine Size vs Price")
plt.show()
```

Q2 - Sports Performance Linear Regression

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_modelimport LinearRegression
from sklearn.model_selectionimport train_test_split
from sklearn.metrics import mean_squared_error
df =pd.DataFrame({
    "Training_Hours":[2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
    "Performance":[40, 45, 50, 55, 60, 65, 70, 75, 82, 88]
})
x =df[["Training_Hours"]]
y =df["Performance"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
model =LinearRegression()
model.fit(x_train, y_train)
pred =model.predict(x_test)
print("MSE:", mean_squared_error(y_test, pred))
plt.scatter(x, y)
plt.plot(x, model.predict(x))
plt.show()
```

ORQ2 - Customer KNN

```
import pandas as pd
from sklearn.model_selectionimport train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
df =pd.DataFrame({
    "Age":[20, 22, 25, 28, 30, 35, 40, 45, 50, 55],
    "Income":[20, 25, 30, 35, 40, 50, 60, 70, 80, 90 ],
    "Class":[0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
})
x =df[["Age", "Income"]]
y =df["Class"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
model =KNeighborsClassifier(n_neighbors=3)
```

```
model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print(confusion_matrix(y_test, pred))
# ============================================================
```

SLIP 7

```
# ============================================================
```

Q1- Course Bar Chart

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]

students = [60, 45, 75, 50, 35]

plt.bar(courses, students)

plt.xlabel("Courses")

plt.ylabel("Students")

plt.title("Students Enrolled")

plt.show()
```

Q2 - Student KNN

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df =pd.DataFrame({

    "Study_Hours":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],

    "Attendance":[50, 55, 60, 65, 70, 75, 80, 85, 90, 95],

    "Performance":[0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
})
x =df[["Study_Hours", "Attendance"]]

y =df["Performance"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
model =KNeighborsClassifier(n_neighbors=3)

model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print(confusion_matrix(y_test, pred))
```

ORQ2 - Groceries Apriori

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

data =[

    ["BasmatiRice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],

    ["Atta", "Sugar", "Tea", "Milk"],

    ["Rice", "Moong Dal", "Salt", "Cooking Oil"],
```

```
    ["Wheat Flour", "Chickpeas", "Turmeric Powder", "CuminSeeds"],

    ["Poha", "Peanuts", "Onions", "Cooking Oil"],

    ["BasmatiRice", "Paneer", "GaramMasala", "Tomatoes"],

    ["Atta", "Potatoes", "Onions", "Cooking Oil"]
]
 te = TransactionEncoder()

 arr = te.fit(data).transform(data)

 df =pd.DataFrame(arr, columns = te.columns_)

 for support in[.3, .4]:

     freq = apriori(df, min_support = support, use_colnames = True)

     print(freq)

     if len(freq)>1:

     print(association_rules(
     freq,

     metric="confidence",

     min_threshold=.5

     ))

 # ============================================================
```

SLIP 8

```
 # ============================================================
```

Q1- Earthquake Geospatial

```
 import pandas as pd

 import matplotlib.pyplot as plt

 df =pd.DataFrame({

     "Latitude":[19.07, 28.61, 18.52, 21.14, 23.02],

     "Longitude":[72.87, 77.20, 73.85, 79.08, 72.57],

     "Depth":[10, 20, 15, 30, 25],

     "Magnitude":[4.2, 5.1, 3.8, 6.0, 4.7]
})
 plt.scatter(

     df["Longitude"],
     df["Latitude"],

     c = df["Magnitude"],

     s = df["Magnitude"]*80
)
 plt.colorbar(label="Magnitude")

 plt.xlabel("Longitude")

 plt.ylabel("Latitude")

 plt.title("Earthquake Locations")

 plt.show()
```

Q2 - LoanData Reduction

```
 import pandas as pd

 from sklearn.preprocessing import LabelEncoder, StandardScaler

 from sklearn.feature_selectionimport SelectKBest, chi2
 from sklearn.decompositionimport PCA
```

```
df =pd.DataFrame({
"Age":[25, 30, 35, 40, 45, 50, 28, 32, 38, 42],
"Income":
[25000, 35000, 45000, 55000, 65000, 75000, 30000, 40000, 50000, 60000 ],
"Credit_Score":[600, 650, 680, 700, 720, 750, 620, 660, 690, 710 ],
"Loan_Amount":
[100000, 150000, 200000, 250000, 300000, 350000, 120000, 180000, 220000,
280000 ],
"Employment_Years":[2, 4, 6, 8, 10, 12, 3, 5, 7, 9],
"Debt_Ratio":[.5, .4, .35, .3, .25, .2, .45, .38, .32, .28],
"Loan_Status":["Rejected", "Rejected", "Approved", "Approved", "Approved",
"Approved", "Rejected", "Approved", "Approved", "Approved"]
})
x =df.drop("Loan_Status", axis=1)

y =LabelEncoder().fit_transform(df["Loan_Status"])

sel =SelectKBest(chi2, k=3)
sel.fit(x, y)
print(x.columns[sel.get_support()].tolist())

x =StandardScaler().fit_transform(x)

pca =PCA(n_components=2)

print(pca.fit_transform(x))
```

ORQ2 - Salary Linear Regression

```
import pandas as pd

import matplotlib.pyplot as plt
from sklearn.model_selectionimport train_test_split

from sklearn.linear_modelimport LinearRegression

from sklearn.metrics import mean_squared_error

df =pd.DataFrame({
"YearsExperience":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Salary":
[30000, 35000, 40000, 45000, 50000, 60000, 65000, 70000, 80000, 90000 ]
})
x =df[["YearsExperience"]]

y =df["Salary"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =LinearRegression()

model.fit(x_train, y_train)

pred =model.predict(x_test)

print("MSE:", mean_squared_error(y_test, pred))

plt.scatter(x, y)

plt.plot(x, model.predict(x))
plt.show()

# ============================================================
```

SLIP 9

```
# ============================================================
```

Q1- Min-Max Scaling

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler

df =pd.DataFrame({
"Price":[100, 200, 300, 400, 500 ],
"Quantity":[10, 20, 30, 40, 50 ],
"Discount":[5, 10, 15, 20, 25],
"Revenue":[950, 3800, 7650, 15200, 23750 ]
})
result =MinMaxScaler().fit_transform(df)

print(pd.DataFrame(result, columns = df.columns))
```

Q2 - Smartphone Logistic Regression

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.linear_modelimport LogisticRegression

from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

df =pd.DataFrame({
"RAM":[2, 3, 4, 4, 6, 6, 8, 8, 12, 12],
"Storage":[32, 32, 64, 64, 128, 128, 256, 256, 256, 512],
"Battery":[3000, 3500, 4000, 4200, 4500, 4800, 5000, 5200, 5500, 6000 ],
"Camera":[8, 10, 12, 16, 20, 24, 32, 40, 48, 50 ],
"Category":[0, 0, 0, 1, 1, 1, 2, 2, 2, 2]
})
x =df.drop("Category", axis=1)

y =df["Category"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =LogisticRegression(max_iter=1000 )

model.fit(x_train, y_train)
pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print("F1:", f1_score(y_test, pred, average="weighted"))

print(confusion_matrix(y_test, pred))
```

ORQ2 - Books Venn+WordCloud

```
import pandas as pd

import matplotlib.pyplot as plt

from matplotlib_vennimport venn2

from wordcloud import WordCloud

df =pd.DataFrame({
"Book_Title":["PythonBasics", "Data Science", "AIGuide",
"Machine Learning", "Deep Learning", "PythonAdvanced"],
"Author":["A", "B", "A", "C", "B", "D"],
"Publication_Year":[2010, 2012, 2015, 2020, 2021, 2022],
"Rating":[4.1, 4.2, 4.3, 4.5, 4.6, 4.7]
})
old = set(df[df["Publication_Year"]<2018]["Author"])
```

```
new = set(df[df["Publication_Year"]>=2018]["Author"])
```
venn2([old, new], set_labels = ("Before 2018", "After 2018"))
```
plt.show()

text="".join(df["Book_Title"])

wc = WordCloud(

    width=800,

    height=400,

    background_color="white"
```
).generate(text)
```
plt.imshow(wc, interpolation="bilinear")

plt.axis("off")

plt.show()

# ============================================================
```

SLIP 10

```
# ============================================================
```

Q1- Transport Pie Chart

```
import matplotlib.pyplot as plt
transport = ["Train", "Bicycle", "Bike", "Car", "Bus"]

employees = [220, 60, 70, 80, 40 ]

plt.pie(

    employees,

    labels =transport,

    autopct="%1.1f%%"
```
)
```
plt.title("Preferred Transport")

plt.show()
```

Q2 - Housing RandomForest Regression

```
import pandas as pd

import numpy as np

from sklearn.model_selectionimport train_test_split

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df =pd.DataFrame({
```
"Area":[500, 600, 700, 800, 900, 1000, 1100, 1200, 1300, 1400],

"Bedrooms":[1, 1, 2, 2, 2, 3, 3, 3, 4, 4],

"Age":[10, 8, 7, 6, 5, 5, 4, 3, 2, 1],

"Price":[20, 25, 30, 35, 40, 48, 55, 60, 70, 80]

})
```
x =df.drop("Price", axis=1)

y =df["Price"]
```
x_train, x_test, y_train, y_test = train_test_split(

x, y, test_size=.2, random_state=42

)
```
model =RandomForestRegressor(

    n_estimators=100,

    random_state=42
```

```
)
 model.fit(x_train, y_train)

 pred =model.predict(x_test)

 print("MAE:", mean_absolute_error(y_test, pred))

 print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))

 print("R2:", r2_score(y_test, pred))
```

ORQ2 - AprioriLearning Resources

```
 import pandas as pd

 from mlxtend.preprocessing import TransactionEncoder

 from mlxtend.frequent_patterns import apriori, association_rules

 data =[

     ["Video Lecture", "Quiz"],

     ["Video Lecture", "Assignment"],

     ["Quiz", "DiscussionForum"],

     ["Video Lecture", "Quiz", "Assignment"],

     ["Assignment", "DiscussionForum"],

     ["Video Lecture", "Certificate"],

     ["Quiz", "Certificate"],

     ["Video Lecture", "Quiz"],

     ["Assignment", "Certificate"],

     ["Video Lecture", "Quiz", "Certificate"]
]
 te = TransactionEncoder()

 arr = te.fit(data).transform(data)

 df =pd.DataFrame(arr, columns = te.columns_)

 for s in[.3, .4]:

     freq = apriori(df, min_support = s, use_colnames = True)

     print(freq)
     if len(freq)>1:

     print(association_rules(

     freq,

     metric="confidence",

     min_threshold=.5

     ))

 # ============================================================
```

SLIP 11

```
 # ============================================================
```

Q1- Department Bar Chart

```
 import matplotlib.pyplot as plt

 departments = ["HR", "Finance", "IT", "Sales", "Marketing"]

 employees = [20, 30, 50, 40, 25]

 plt.bar(departments, employees)
 plt.xlabel("Department")
```

```
plt.ylabel("Employees")

plt.title("Employees by Department")

plt.show()
```

Q2 - Smartphone Logistic Regression

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.linear_modelimport LogisticRegression

from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

df =pd.DataFrame({
"RAM":[2, 3, 4, 4, 6, 6, 8, 8, 12, 12],
"Storage":[32, 32, 64, 64, 128, 128, 256, 256, 256, 512],
"Battery":[3000, 3500, 4000, 4200, 4500, 4800, 5000, 5200, 5500, 6000 ],
"Camera":[8, 10, 12, 16, 20, 24, 32, 40, 48, 50 ],
"Category":[0, 0, 0, 1, 1, 1, 2, 2, 2, 2]
})
x =df.drop("Category", axis=1)

y =df["Category"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =LogisticRegression(max_iter=1000 )
model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print("F1:", f1_score(y_test, pred, average="weighted"))

print(confusion_matrix(y_test, pred))
```

ORQ2 - E-Commerce K-Means

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

df =pd.DataFrame({
"Income":[15, 20, 25, 30, 35, 50, 55, 60, 70, 80 ],
"Spending":[80, 75, 70, 65, 60, 45, 40, 35, 25, 20 ]
})
model =KMeans(n_clusters=3, random_state=42, n_init=10 )

df["Cluster"] = model.fit_predict(

    df[["Income", "Spending"]]
)
print(df)

plt.scatter(

    df["Income"],

    df["Spending"],

    c = df["Cluster"]
)
plt.show()

# ============================================================
```

SLIP 12

```
# ============================================================
```

Q1- Titanic Histogram+Pie

```
import pandas as pd

import matplotlib.pyplot as plt

df =pd.DataFrame({
"Pclass":[1, 1, 2, 2, 3, 3, 3, 1, 2, 3],
"Survived":[1, 1, 1, 0, 0, 1, 0, 1, 0, 1]
})
plt.hist(df["Pclass"], bins = [.5, 1.5, 2.5, 3.5])

plt.xlabel("Passenger Class")

plt.ylabel("Passengers")

plt.title("Passenger Class")

plt.show()

df["Survived"].value_counts().plot.pie(

    labels =["Not Survived", "Survived"],

    autopct="%1.1f%%"
)
plt.ylabel("")

plt.show()
```

Q2 - PolynomialRegression

```
import pandas as pd

import numpy as np

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_modelimport LinearRegression

from sklearn.model_selectionimport train_test_split

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
df =pd.DataFrame({
"Danceability":[.2, .3, .4, .5, .6, .7, .8, .9, .95, .85],
"Energy":[.3, .4, .5, .6, .7, .75, .8, .85, .9, .65],
"Tempo":[80, 90, 100, 110, 120, 125, 130, 135, 140, 115],
"Popularity":[30, 35, 40, 45, 52, 60, 65, 72, 80, 58]
})
x =df[["Danceability", "Energy", "Tempo"]]

y =df["Popularity"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
poly = PolynomialFeatures(degree=2)

x_train = poly.fit_transform(x_train)

x_test = poly.transform(x_test)

model =LinearRegression()

model.fit(x_train, y_train)

pred =model.predict(x_test)
print("MAE:", mean_absolute_error(y_test, pred))

print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2:", r2_score(y_test, pred))
```

ORQ2 - Spambase KNN

```
from sklearn.datasets import make_classification

from sklearn.model_selectionimport train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix
x, y =make_classification(
    n_samples=300,

    n_features=10,

    random_state=42
)
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=.2, random_state=42
)
scaler =StandardScaler()
x_train = scaler.fit_transform(x_train)

x_test = scaler.transform(x_test)

model =KNeighborsClassifier(n_neighbors=5)

model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

print(confusion_matrix(y_test, pred))

# ============================================================
```

SLIP 13

```
# ============================================================
```

Q1- Sales Box Plot

```
import pandas as pd

import matplotlib.pyplot as plt

df =pd.DataFrame({
"Sales":[120, 150, 180, 200, 220, 250, 270, 300, 350, 700 ]
})
plt.boxplot(df["Sales"])

plt.title("Sales Box Plot")

plt.ylabel("Sales")

plt.show()
```

Q2 - Wine KNN

```
from sklearn.datasets import load_wine

from sklearn.model_selectionimport train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

wine = load_wine()

x =wine.data

y =wine.target
x_train, x_test, y_train, y_test = train_test_split(
```

```
x, y, test_size=.2, random_state=42
)
 scaler =StandardScaler()
 x_train = scaler.fit_transform(x_train)

 x_test = scaler.transform(x_test)

 model =KNeighborsClassifier(n_neighbors=5)

 model.fit(x_train, y_train)

 pred =model.predict(x_test)

 print("Accuracy:", accuracy_score(y_test, pred))
```

ORQ2 - Online Gaming Logistic Regression

```
 import pandas as pd

 from sklearn.model_selectionimport train_test_split

 from sklearn.linear_modelimport LogisticRegression

 from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

 df =pd.DataFrame({
"Play_Time":[1, 2, 2, 3, 4, 5, 6, 7, 8, 9],
"Achievements":[2, 3, 4, 5, 6, 7, 8, 9, 10, 12],
"Purchases":[0, 1, 0, 1, 2, 2, 3, 3, 4, 5],
"Level":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Class":[0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
})
 x =df.drop("Class", axis=1)

 y =df["Class"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
 model =LogisticRegression()

 model.fit(x_train, y_train)

 pred =model.predict(x_test)

 print("Accuracy:", accuracy_score(y_test, pred))

 print("Precision:", precision_score(y_test, pred, zero_division=0 ))

 print("Recall:", recall_score(y_test, pred, zero_division=0 ))

 print("F1:", f1_score(y_test, pred, zero_division=0 ))

 # ============================================================
```

SLIP 14

```
 # ============================================================
```

Q1- Product Dendrogram

```
 import pandas as pd

 import matplotlib.pyplot as plt

 from scipy.cluster.hierarchy import dendrogram, linkage

 df =pd.DataFrame({
"Price":[100, 120, 500, 520, 110 ],
"Rating":[4.1, 4.0, 4.8, 4.7, 4.2],
"Sales":[90, 85, 35, 40, 95]
})
 z =linkage(df, method="ward")
```

dendrogram(z)

```
plt.title("Product Dendrogram")

plt.show()
```

Q2 - Car Price PolynomialRegression

```
import pandas as pd

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_modelimport LinearRegression

from sklearn.model_selectionimport train_test_split
from sklearn.metrics import r2_score

df =pd.DataFrame({
```

"Vehicle_Age":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

"Present_Price":[8, 7.5, 7, 6.5, 6, 5.5, 5, 4.5, 4, 3.5],

"Kilometers_Driven":[10000, 15000, 20000, 25000, 30000,

35000, 40000, 45000, 50000, 55000],

"Selling_Price":[7, 6.5, 6, 5.5, 5, 4.5, 4, 3.5, 3, 2.5]

})

```
x =df[["Vehicle_Age", "Present_Price", "Kilometers_Driven"]]

y =df["Selling_Price"]
```

x_train, x_test, y_train, y_test = train_test_split(

x, y, test_size=.2, random_state=42

)

```
poly = PolynomialFeatures(degree=2)

x_train = poly.fit_transform(x_train)

x_test = poly.transform(x_test)

model =LinearRegression()

model.fit(x_train, y_train)
pred =model.predict(x_test)

print("R2:", r2_score(y_test, pred))
```

ORQ2 - Mobile Data Integration+Encoding

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
```

"Brand":["Samsung", "Apple", "OnePlus", "Vivo"],

"Model":["A", "B", "C", "D"],

"Price":[20000, 70000, 35000, 25000],

"RAM":[6, 8, 8, 6]

})

```
df2 = pd.DataFrame({
```

"Model":["A", "B", "C", "D"],

"Storage":[128, 256, 128, 128],

"Battery_Capacity":[5000, 3500, 4500, 4800],

"Operating_System":["Android", "iOS", "Android", "Android"]

})

```
df =pd.merge(df1, df2, on="Model")
le = LabelEncoder()

df["Brand"] = le.fit_transform(df["Brand"])

df =pd.get_dummies(df, columns = ["Operating_System"])

print(df)
```

```
# ============================================================
```

SLIP 15

```
# ============================================================
```

Q1- EqualFrequency Binning

```
import pandas as pd

df =pd.DataFrame({
"Person_ID":range(1, 11),
"Age":[18, 22, 25, 30, 35, 40, 45, 50, 55, 60 ]
})
df["Age_Bin"] = pd.qcut(

    df["Age"],
    q=3,

    labels =["Low", "Medium", "High"]
)
print(df)
```

Q2 - City Treemap +3DScatter

```
import pandas as pd

import matplotlib.pyplot as plt

import squarify

df =pd.DataFrame({
"City":["Mumbai", "Delhi", "Pune", "Nashik", "Nagpur"],
"Population":[20, 19, 7, 2, 3],
"GDP":[400, 350, 150, 60, 80 ],
"Literacy":[89, 88, 90, 82, 88]
})
squarify.plot(
    sizes = df["Population"],

    label = df["City"]
)
plt.title("City PopulationTreemap")

plt.axis("off")

plt.show()

fig = plt.figure()

ax = fig.add_subplot(111, projection="3d")
ax.scatter(
    df["Population"],

    df["GDP"],

    df["Literacy"]
)
ax.set_xlabel("Population")
ax.set_ylabel("GDP")
ax.set_zlabel("Literacy")
plt.show()
```

ORQ2 - Groceries Apriori

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules
data =[
```

["BasmatiRice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],
["Atta", "Sugar", "Tea", "Milk"],
["Rice", "Moong Dal", "Salt", "Cooking Oil"],
["Wheat Flour", "Chickpeas", "Turmeric Powder", "CuminSeeds"],
["Poha", "Peanuts", "Onions", "Cooking Oil"],
["BasmatiRice", "Paneer", "GaramMasala", "Tomatoes"],
["Atta", "Potatoes", "Onions", "Cooking Oil"]
]

```
te = TransactionEncoder()

arr = te.fit(data).transform(data)

df =pd.DataFrame(arr, columns = te.columns_)

for s in[.3, .4]:

    freq = apriori(df, min_support = s, use_colnames = True)

    print("Support:", s)

    print(freq)

    if len(freq)>1:

    print(association_rules(

    freq,

    metric="confidence",
    min_threshold=.5

    ))

# ============================================================
```

SLIP 16

```
# ============================================================
```

Q1- Student CorrelationHeatmap

```
import pandas as pd

import seabornas sns

import matplotlib.pyplot as plt

df =pd.DataFrame({
```

"Study_Hours":[2, 3, 4, 5, 6, 7, 8, 9],
"Attendance":[60, 65, 70, 75, 80, 85, 90, 95],
"Assignments":[3, 4, 5, 6, 7, 8, 9, 10],
"Marks":[40, 45, 50, 58, 65, 72, 80, 90]
})

```
sns.heatmap(df.corr(), annot = True)

plt.title("Student CorrelationHeatmap")
plt.show()
```

Q2 - Data Transformations

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df =pd.DataFrame({
```

"Temperature":[20, 25, 30, 35, 40],
"Humidity":[40, 50, 60, 70, 80],
"Wind_Speed":[5, 10, 15, 20, 25],
"Pressure":[1000, 1005, 1010, 1015, 1020],
"Rainfall":[10, 20, 30, 40, 50]

```
})
print("Min-Max:")
print(MinMaxScaler().fit_transform(df))
print("Standardization:")
print(StandardScaler().fit_transform(df))
print("Normalization:")
print(Normalizer().fit_transform(df))
```

ORQ2 - Student Activity Apriori

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
data =[
["Video", "Quiz"],
["Video", "Assignment"],
["Quiz", "Discussion"],
["Video", "Quiz", "Assignment"],
["Assignment", "Discussion"],
["Video", "Certificate"],
["Quiz", "Certificate"],
["Video", "Quiz"],
["Assignment", "Certificate"],
["Video", "Quiz", "Certificate"]
]
te = TransactionEncoder()
arr = te.fit(data).transform(data)
df =pd.DataFrame(arr, columns = te.columns_)
freq = apriori(df, min_support=.3, use_colnames = True)
print(freq)
print(association_rules(
    freq,
    metric="confidence",
    min_threshold=.5
))
# ============================================================
```

SLIP 17

```
# ============================================================
```

Q1- Student Data Integration

```
import pandas as pd
df1 = pd.DataFrame({
"Student_ID":[1, 2, 3, 4],
"Name":["A", "B", "C", "D"],
"Age":[18, 19, 18, 20 ]
})
df2 = pd.DataFrame({
"Student_ID":[1, 2, 3, 4],
"Marks":[75, 80, 65, 90 ],
"Attendance":[80, 85, 75, 95]
```

```
})
df =pd.merge(df1, df2, on="Student_ID")
print(df)
```

Q2 - City Treemap +3DScatter

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify
df =pd.DataFrame({
"City":["Mumbai", "Delhi", "Pune", "Nashik", "Nagpur"],
"Population":[20, 19, 7, 2, 3],
"GDP":[400, 350, 150, 60, 80 ],
"Literacy":[89, 88, 90, 82, 88]
})
squarify.plot(
    sizes = df["Population"],
    label = df["City"]
)
plt.title("City PopulationTreemap")
plt.axis("off")
plt.show()
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.scatter(
    df["Population"],
    df["GDP"],
    df["Literacy"]
)
plt.show()
```

ORQ2 - Student Grading RandomForest

```
import pandas as pd
from sklearn.model_selectionimport train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
df =pd.DataFrame({
"Study_Hours":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Attendance":[50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
"Assignments":[2, 3, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Performance":[0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
})
x =df.drop("Performance", axis=1)
y =df["Performance"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
model.fit(x_train, y_train)

pred =model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, pred))

# ============================================================
```

SLIP 18

```
# ============================================================
```

Q1- Employee Cleaning +IQR

```
import pandas as pd

df =pd.DataFrame({
"Emp_ID":[1, 2, 3, 3, 4, 5],
"Age":[25, 30, None, 35, 40, 100 ],
"Dept":["IT", "HR", "IT", "IT", "Sales", "HR"],
"Experience":[2, 5, 3, 3, 8, 10 ],
"Rating":[3, 4, 3, 3, 5, 4]
})
df =df.drop_duplicates()

df["Age"] = df["Age"].fillna(df["Age"].median())
Q1 = df["Age"].quantile(.25)

Q3 = df["Age"].quantile(.75)

IQR = Q3-Q1

lower = Q1-1.5*IQR

upper = Q3+1.5*IQR

outliers = df[

    (df["Age"]<lower) |

    (df["Age"]>upper)
]
print("Cleaned Data:")

print(df)

print("Outliers:")

print(outliers)
```

Q2 - World Cities Geospatial+Treemap

```
import pandas as pd

import matplotlib.pyplot as plt

import squarify
df =pd.DataFrame({
"City":["Mumbai", "Delhi", "Pune", "Nashik", "Nagpur"],
"Latitude":[19.07, 28.61, 18.52, 19.99, 21.14],
"Longitude":[72.87, 77.20, 73.85, 73.79, 79.08],
"Population":[20, 19, 7, 2, 3]
})
plt.scatter(

    df["Longitude"],

    df["Latitude"],

    s = df["Population"]*100
)
plt.xlabel("Longitude")

plt.ylabel("Latitude")
```

```
plt.title("World Cities")

plt.show()
```
squarify.plot(
```
    sizes = df["Population"],

    label = df["City"]
```
)
```
plt.axis("off")

plt.show()
```
ORQ2 - Crop Yield Linear Regression
```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selectionimport train_test_split

from sklearn.linear_modelimport LinearRegression

from sklearn.metrics import mean_absolute_error, r2_score

df =pd.DataFrame({
```
"Rainfall":[500, 550, 600, 650, 700, 750, 800, 850, 900, 950],

"Yield":[20, 23, 25, 28, 30, 33, 35, 38, 40, 43]

})
```
x =df[["Rainfall"]]

y =df["Yield"]
```
x_train, x_test, y_train, y_test = train_test_split(

x, y, test_size=.2, random_state=42

)
```
model =LinearRegression()

model.fit(x_train, y_train)
pred =model.predict(x_test)

print("MAE:", mean_absolute_error(y_test, pred))

print("R2:", r2_score(y_test, pred))

plt.scatter(x, y)

plt.plot(x, model.predict(x))

plt.show()

# ============================================================
```
SLIP 19
```
# ============================================================
```
Q1- Vehicle Box Plots
```
import pandas as pd

import matplotlib.pyplot as plt

df =pd.DataFrame({
```
"Fuel_Efficiency":[20, 22, 24, 25, 60],

"Engine_Power":[100, 120, 130, 140, 300],

"Vehicle_Weight":[1200, 1300, 1400, 1500, 1600],

"Engine_Size":[1000, 1200, 1400, 1600, 1800],

"Acceleration":[12, 11, 10, 9, 8]

})
```
for colindf.columns:

    plt.boxplot(df[col])
    plt.title(col)
```

```
    plt.show()
```

Q2 - Data Transformations

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df =pd.DataFrame({
"Temperature":[20, 25, 30, 35, 40 ],
"Humidity":[40, 50, 60, 70, 80 ],
"Wind_Speed":[5, 10, 15, 20, 25],
"Pressure":[1000, 1005, 1010, 1015, 1020 ],
"Rainfall":[10, 20, 30, 40, 50 ]
})
print(MinMaxScaler().fit_transform(df))

print(StandardScaler().fit_transform(df))

print(Normalizer().fit_transform(df))
```

ORQ2 - Heart Disease RandomForest

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

df =pd.DataFrame({
"Age":[45, 50, 55, 60, 65, 40, 48, 52, 58, 62],
"Cholesterol":[200, 220, 250, 270, 290, 180, 210, 230, 260, 280 ],
"BP":[120, 130, 140, 150, 160, 110, 125, 135, 145, 155],
"HeartDisease":[0, 0, 1, 1, 1, 0, 0, 1, 1, 1]
})
x =df.drop("HeartDisease", axis=1)

y =df["HeartDisease"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =RandomForestClassifier(

    n_estimators=100,
    random_state=42
)
model.fit(x_train, y_train)

pred =model.predict(x_test)

print("Accuracy:", accuracy_score(y_test, pred))

# ============================================================
```

SLIP 20

```
# ============================================================
```

Q1- Standardization

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

df =pd.DataFrame({
"Age":[20, 25, 30, 35, 40 ],
"Salary":[20000, 30000, 40000, 50000, 60000 ],
"Experience":[1, 3, 5, 7, 9],
```

```
"Working_Hours":[6, 7, 8, 9, 10 ],
"Performance":[50, 60, 70, 80, 90 ]
})
 result =StandardScaler().fit_transform(df)
 print(pd.DataFrame(result, columns = df.columns))
```

Q2 - Books Venn+WordCloud

```
 import pandas as pd
 import matplotlib.pyplot as plt
 from matplotlib_vennimport venn2
 from wordcloud import WordCloud
 df =pd.DataFrame({
"Book_Title":["PythonBasics", "Data Science", "AIGuide",
"Machine Learning", "Deep Learning", "PythonAdvanced"],
"Author":["A", "B", "A", "C", "B", "D"],
"Publication_Year":[2010, 2012, 2015, 2020, 2021, 2022]
})
 old = set(df[df["Publication_Year"]<2018]["Author"])
 new = set(df[df["Publication_Year"]>=2018]["Author"])
venn2([old, new], set_labels = ("Before 2018", "After 2018"))
 plt.show()
 text="".join(df["Book_Title"])
 wc = WordCloud(
     width=800,
     height=400,
     background_color="white"
).generate(text)
 plt.imshow(wc, interpolation="bilinear")
 plt.axis("off")
 plt.show()
```

ORQ2 - Student Activity Apriori

```
 import pandas as pd
 from mlxtend.preprocessing import TransactionEncoder
 from mlxtend.frequent_patterns import apriori, association_rules
 data =[
["Video", "Quiz"],
["Video", "Assignment"],
["Quiz", "Discussion"],
["Video", "Quiz", "Assignment"],
["Assignment", "Discussion"],
["Video", "Certificate"],
["Quiz", "Certificate"],
["Video", "Quiz"],
["Assignment", "Certificate"],
["Video", "Quiz", "Certificate"]
]
 te = TransactionEncoder()
 arr = te.fit(data).transform(data)
```

```
df =pd.DataFrame(arr, columns = te.columns_)

freq = apriori(df, min_support=.3, use_colnames = True)

print(freq)

print(association_rules(

    freq,

    metric="confidence",

    min_threshold=.5
))
# ============================================================
```

SLIP 21

```
# ============================================================
```

Q1- Weather Heatmap

```
import pandas as pd

import seabornas sns

import matplotlib.pyplot as plt

df =pd.DataFrame({
"Monday":[25, 26, 27, 28, 29],
"Tuesday":[24, 25, 26, 27, 28],
"Wednesday":[23, 25, 27, 29, 30 ],
"Thursday":[22, 24, 26, 28, 29],
"Friday":[21, 23, 25, 27, 28]
})
sns.heatmap(df, annot = True)

plt.title("Weather Heatmap")

plt.show()
```

Q2 - Student K-Means K=3

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

df =pd.DataFrame({
"Study_Hours":[1, 2, 2, 3, 4, 5, 6, 7, 8, 9],
"Marks":[35, 40, 42, 48, 55, 60, 68, 75, 82, 90 ]
})
x =df[["Study_Hours", "Marks"]]

model =KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10
)
df["Cluster"] = model.fit_predict(x)

print(df)

plt.scatter(
    df["Study_Hours"],

    df["Marks"],

    c = df["Cluster"]
)
```

```
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Student K-Means")
plt.show()
print("Centroids:")
print(model.cluster_centers_)
```

ORQ2 - LoanData Reduction

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.feature_selectionimport SelectKBest, chi2
from sklearn.decompositionimport PCA
df =pd.DataFrame({
"Age":[25, 30, 35, 40, 45, 50, 28, 32, 38, 42],
"Income":
[25000, 35000, 45000, 55000, 65000, 75000, 30000, 40000, 50000, 60000 ],
"Credit_Score":[600, 650, 680, 700, 720, 750, 620, 660, 690, 710 ],
"Loan_Amount":
[100000, 150000, 200000, 250000, 300000, 350000, 120000, 180000, 220000,
280000 ],
"Employment_Years":[2, 4, 6, 8, 10, 12, 3, 5, 7, 9],
"Debt_Ratio":[.5, .4, .35, .3, .25, .2, .45, .38, .32, .28],
"Loan_Status":["Rejected", "Rejected", "Approved", "Approved", "Approved",
"Approved", "Rejected", "Approved", "Approved", "Approved"]
})
x =df.drop("Loan_Status", axis=1)
y =LabelEncoder().fit_transform(df["Loan_Status"])
sel =SelectKBest(chi2, k=3)
sel.fit(x, y)
print("Important Features:")
print(x.columns[sel.get_support()].tolist())
x =StandardScaler().fit_transform(x)
pca =PCA(n_components=2)
print(pca.fit_transform(x))
# ============================================================
```

SLIP 22

```
# ============================================================
```

Q1- House Rent Normalization

```
import pandas as pd
from sklearn.preprocessing import Normalizer
df =pd.DataFrame({
"Area":[500, 700, 900, 1100, 1300 ],
"Rent":[10000, 15000, 20000, 25000, 30000 ]
})
result =Normalizer().fit_transform(df)
print("Original:")
print(df)
```

```
print("Normalized:")

print(result)
```

Q2 - Netflix/ AmazonVenn+Pie

```
import matplotlib.pyplot as plt

from matplotlib_vennimport venn2

netflix={"A", "B", "C", "D", "E"}

amazon={"C", "D", "E", "F", "G"}

a = len(netflix-amazon)

b = len(amazon-netflix)

c = len(netflix&amazon)
venn2(
    [netflix, amazon],

    set_labels = ("Netflix", "Amazon")
)
plt.show()

plt.pie(

    [a, b, c],

    labels =["Netflix", "Amazon", "Both"],

    autopct="%1.1f%%"
)
plt.show()
```

ORQ2 - Student K-Means

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

df =pd.DataFrame({
"Study_Hours":[1, 2, 2, 3, 4, 5, 6, 7, 8, 9],
"Marks":[35, 40, 42, 48, 55, 60, 68, 75, 82, 90 ]
})
model =KMeans(

    n_clusters=3,

    random_state=42,

    n_init=10
)
df["Cluster"] = model.fit_predict(

    df[["Study_Hours", "Marks"]]
)
print(df)

plt.scatter(
    df["Study_Hours"],

    df["Marks"],

    c = df["Cluster"]
)
plt.show()

# ============================================================
```

SLIP 23

```
# ============================================================
```

Q1- Product Dendrogram

```
import pandas as pd

import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

df =pd.DataFrame({
"Price":[100, 120, 500, 520, 110 ],
"Rating":[4.1, 4.0, 4.8, 4.7, 4.2],
"Sales":[90, 85, 35, 40, 95]
})
z =linkage(df, method="ward")
dendrogram(z)
plt.title("Product Dendrogram")

plt.show()
```

Q2 - Data Transformations

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df =pd.DataFrame({
"Temperature":[20, 25, 30, 35, 40 ],
"Humidity":[40, 50, 60, 70, 80 ],
"Wind_Speed":[5, 10, 15, 20, 25],
"Pressure":[1000, 1005, 1010, 1015, 1020 ],
"Rainfall":[10, 20, 30, 40, 50 ]
})
print("Min-Max:")

print(MinMaxScaler().fit_transform(df))

print("Standardization:")
print(StandardScaler().fit_transform(df))

print("Normalization:")

print(Normalizer().fit_transform(df))
```

ORQ2 - Student Grading KNN

```
import pandas as pd

from sklearn.model_selectionimport train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

df =pd.DataFrame({
"Study_Hours":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Attendance":[50, 55, 60, 65, 70, 75, 80, 85, 90, 95],
"Grade":[0, 0, 0, 0, 1, 1, 1, 1, 1, 1]
})
x =df[["Study_Hours", "Attendance"]]

y =df["Grade"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
model =KNeighborsClassifier(n_neighbors=3)

model.fit(x_train, y_train)

pred =model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

```
print("Predicted:", pred)

# ============================================================
```

SLIP 24

```
# ============================================================
```

Q1- BookPrice EqualWidthBinning

```
import pandas as pd

df =pd.DataFrame({
"Book_ID":range(1, 11),
"Price":[100, 150, 200, 250, 300, 350, 400, 450, 500, 550 ]
})
df["Price_Bin"] = pd.cut(
    df["Price"],

    bins=3,

    labels =["Low", "Medium", "High"]
)
print(df)
```

Q2 - World Cities Geospatial+Treemap

```
import pandas as pd

import matplotlib.pyplot as plt

import squarify

df =pd.DataFrame({
"City":["Mumbai", "Delhi", "Pune", "Nashik", "Nagpur"],
"Latitude":[19.07, 28.61, 18.52, 19.99, 21.14],
"Longitude":[72.87, 77.20, 73.85, 73.79, 79.08],
"Population":[20, 19, 7, 2, 3]
})
plt.scatter(

    df["Longitude"],

    df["Latitude"],

    s = df["Population"]*100
)
plt.xlabel("Longitude")

plt.ylabel("Latitude")

plt.title("City Locations")

plt.show()
squarify.plot(
    sizes = df["Population"],

    label = df["City"]
)
plt.title("PopulationTreemap")

plt.axis("off")

plt.show()
```

ORQ2 - Car Price PolynomialRegression

```
import pandas as pd

from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_modelimport LinearRegression
from sklearn.model_selectionimport train_test_split
```

```
from sklearn.metrics import r2_score
df =pd.DataFrame({
"Vehicle_Age":[1, 2, 3, 4, 5, 6, 7, 8, 9, 10 ],
"Present_Price":[8, 7.5, 7, 6.5, 6, 5.5, 5, 4.5, 4, 3.5],
"Kilometers_Driven":[10000, 15000, 20000, 25000, 30000,
35000, 40000, 45000, 50000, 55000 ],
"Selling_Price":[7, 6.5, 6, 5.5, 5, 4.5, 4, 3.5, 3, 2.5]
})
x =df[["Vehicle_Age", "Present_Price", "Kilometers_Driven"]]

y =df["Selling_Price"]
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=.2, random_state=42
)
poly = PolynomialFeatures(degree=2)

x_train = poly.fit_transform(x_train)

x_test = poly.transform(x_test)

model =LinearRegression()

model.fit(x_train, y_train)

pred =model.predict(x_test)
print("R2:", r2_score(y_test, pred))

# ============================================================
```

SLIP 25

```
# ============================================================
```

Q1- Course Bar Chart

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]

students = [60, 45, 75, 50, 35]

plt.bar(courses, students)

plt.xlabel("Courses")

plt.ylabel("Students")

plt.title("Students Enrolled inCourses")

plt.show()
```

Q2 - Mobile Data Integration+Encoding

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
"Brand":["Samsung", "Apple", "OnePlus", "Vivo"],
"Model":["A", "B", "C", "D"],
"Price":[20000, 70000, 35000, 25000 ],
"RAM":[6, 8, 8, 6]
})
df2 = pd.DataFrame({
"Model":["A", "B", "C", "D"],
"Storage":[128, 256, 128, 128],
"Battery_Capacity":[5000, 3500, 4500, 4800 ],
"Operating_System":["Android", "iOS", "Android", "Android"]
})
```

```
merged = pd.merge(df1, df2, on="Model")

le = LabelEncoder()

merged["Brand"] = le.fit_transform(merged["Brand"])

merged = pd.get_dummies(

    merged,
    columns = ["Operating_System"]
)
print("Merged and Encoded Data:")

print(merged)
```

ORQ2 - Groceries Apriori

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

data =[
["BasmatiRice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],
["Atta", "Sugar", "Tea", "Milk"],
["Rice", "Moong Dal", "Salt", "Cooking Oil"],
["Wheat Flour", "Chickpeas", "Turmeric Powder", "CuminSeeds"],
["Poha", "Peanuts", "Onions", "Cooking Oil"],
["BasmatiRice", "Paneer", "GaramMasala", "Tomatoes"],
["Atta", "Potatoes", "Onions", "Cooking Oil"]
]
te = TransactionEncoder()

arr = te.fit(data).transform(data)
df =pd.DataFrame(

    arr,

    columns = te.columns_
)
for support in[.3, .4]:

    print("\ nMinimumSupport:", support)

    frequent = apriori(

    df,

    min_support = support,

    use_colnames = True

    )

    print("Frequent Itemsets:")

    print(frequent)

    if len(frequent)>1:

    rules = association_rules(

    frequent,

    metric="confidence",

    min_threshold=.5
    )

    print("AssociationRules:")

    print(rules)

# ============================================================
```

```
# END- SLIP 1TOSLIP 25

# ============================================================
```

□□ —□□□ □□ □□□□ code block

```
    □ □□ □□□, □□□□□□ □□□ block□□□□ copy□    □□□□□.
```

□ □ □□ □□□ : OR□□□□ exam□ □□ □□□ option□□□□□□□□, □□ □□ □□□□.